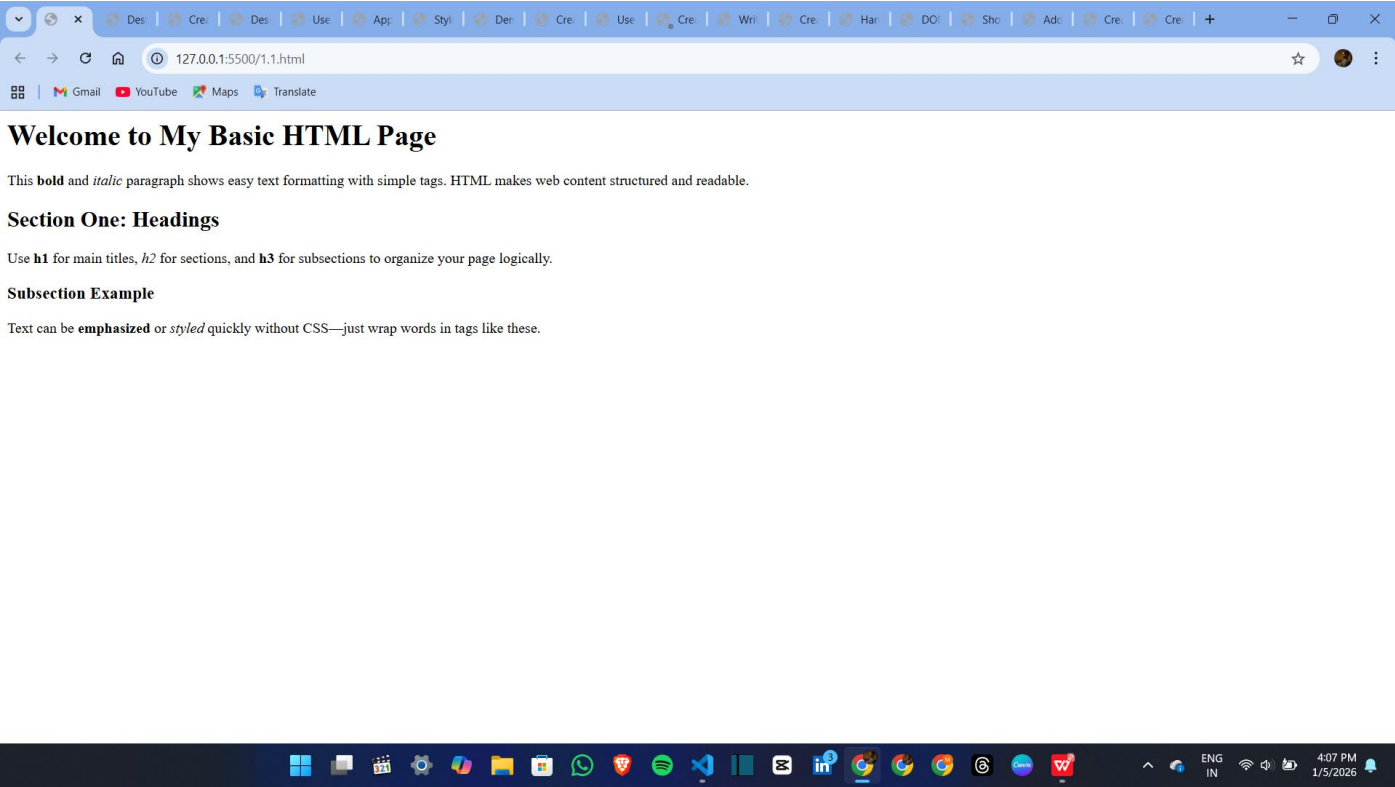
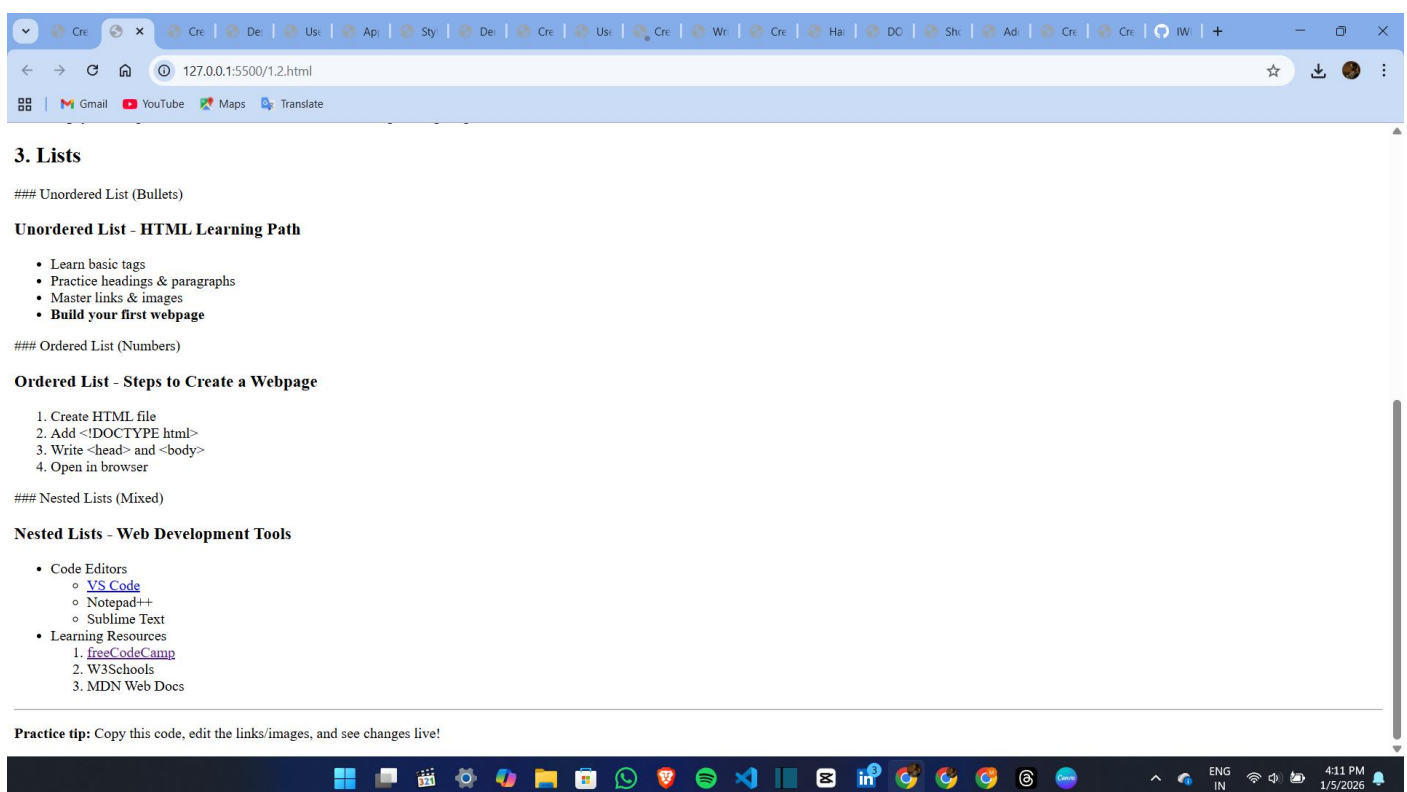
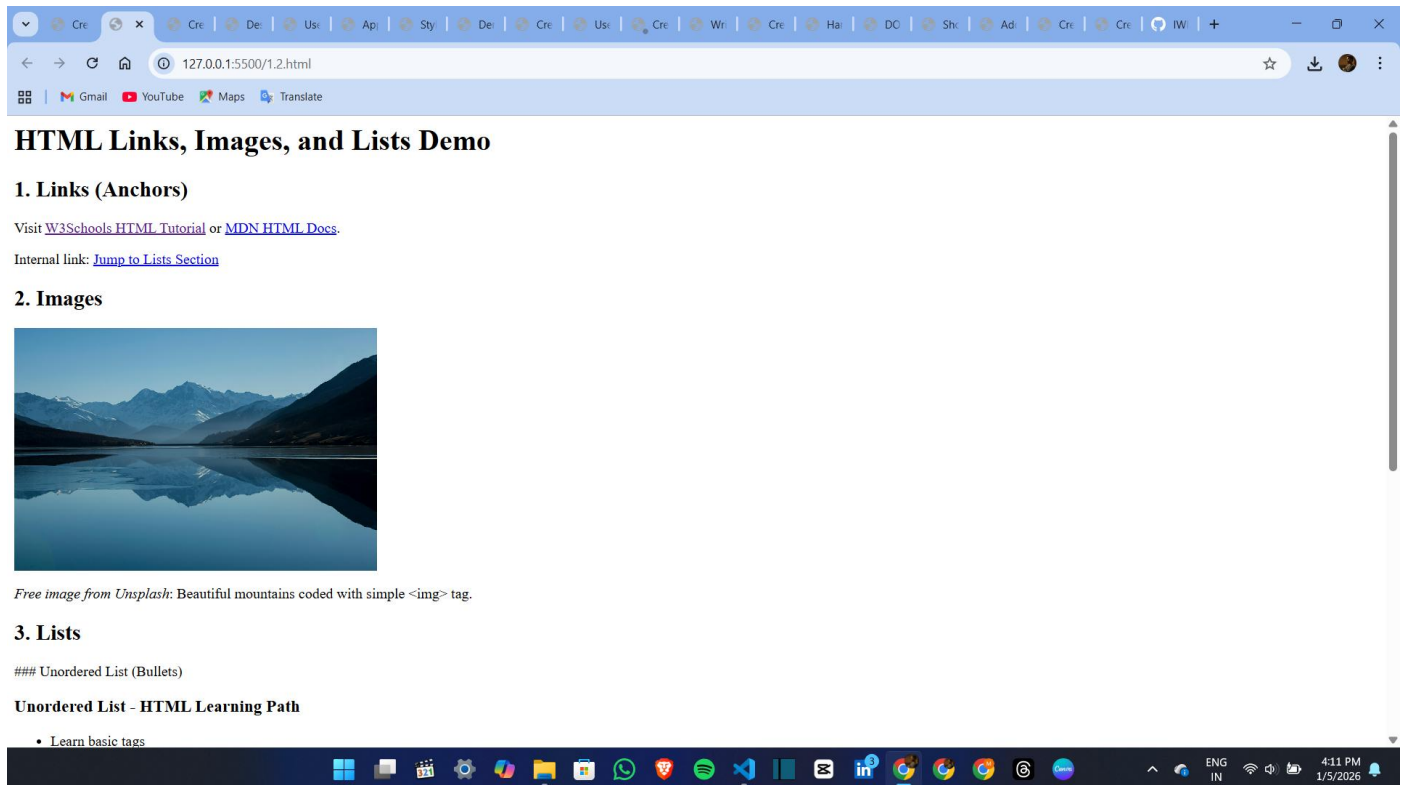


1. Create a basic HTML web page using headings, paragraphs, and text formatting tags



2. Design a webpage demonstrating HTML links, images, and lists (ordered, unordered, nested)



3. Create a table-based layout with merged cells, alignment, and caption using HTML

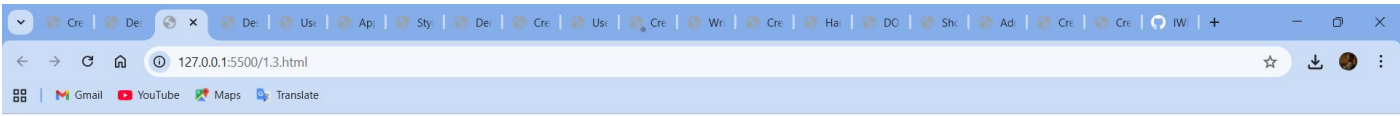


Table-Based Layout Demo

Student Grade Report - Semester 2025					
Student ID	Subjects			Total	Grade
	Math	Science	English		
C001	95	88	92	275	A+
C002	82	85 Programming (90)	78	245	A
C003			87	262	A
C004	76	82	89	247	B+

Layout Breakdown

- `rowspan="2"`: Merges 2 rows vertically (Student ID, Total, Grade headers)
- `colspan="3"`: Merges 3 columns horizontally (Subjects header)
- `align="center/right/left"`: Text alignment in cells
- `<caption>`: Table title above

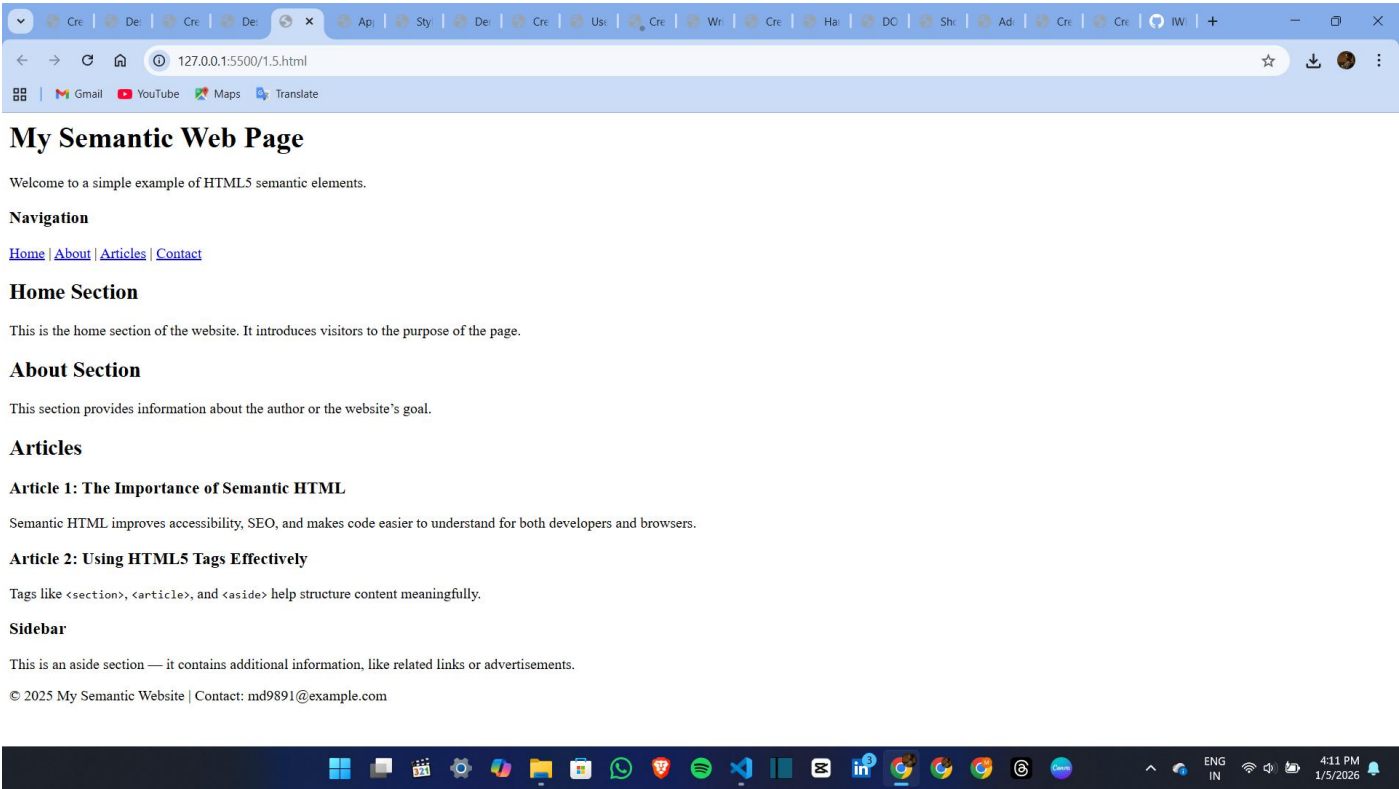


4. Design a webpage with an HTML form having text input, radio buttons, checkboxes, select menus, and submit/reset buttons

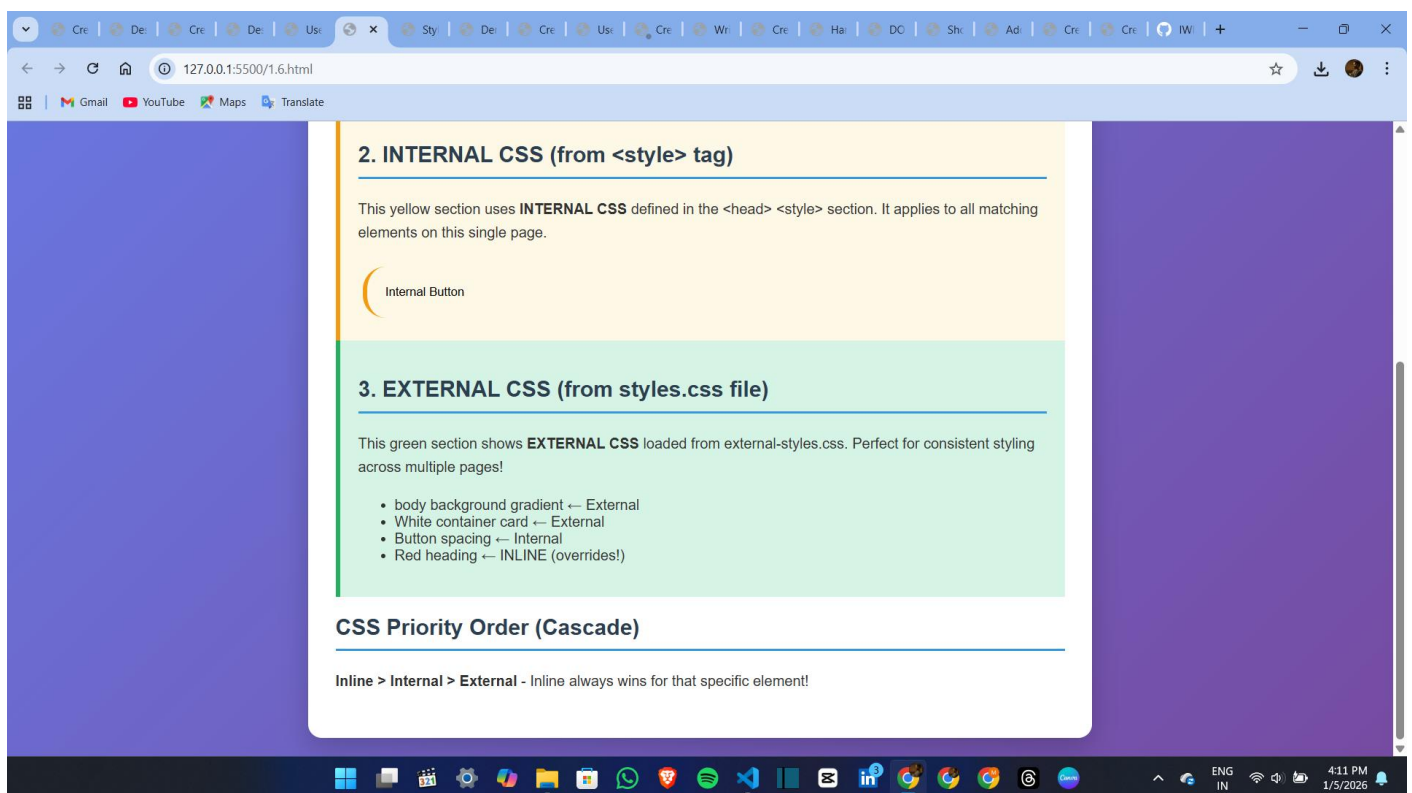
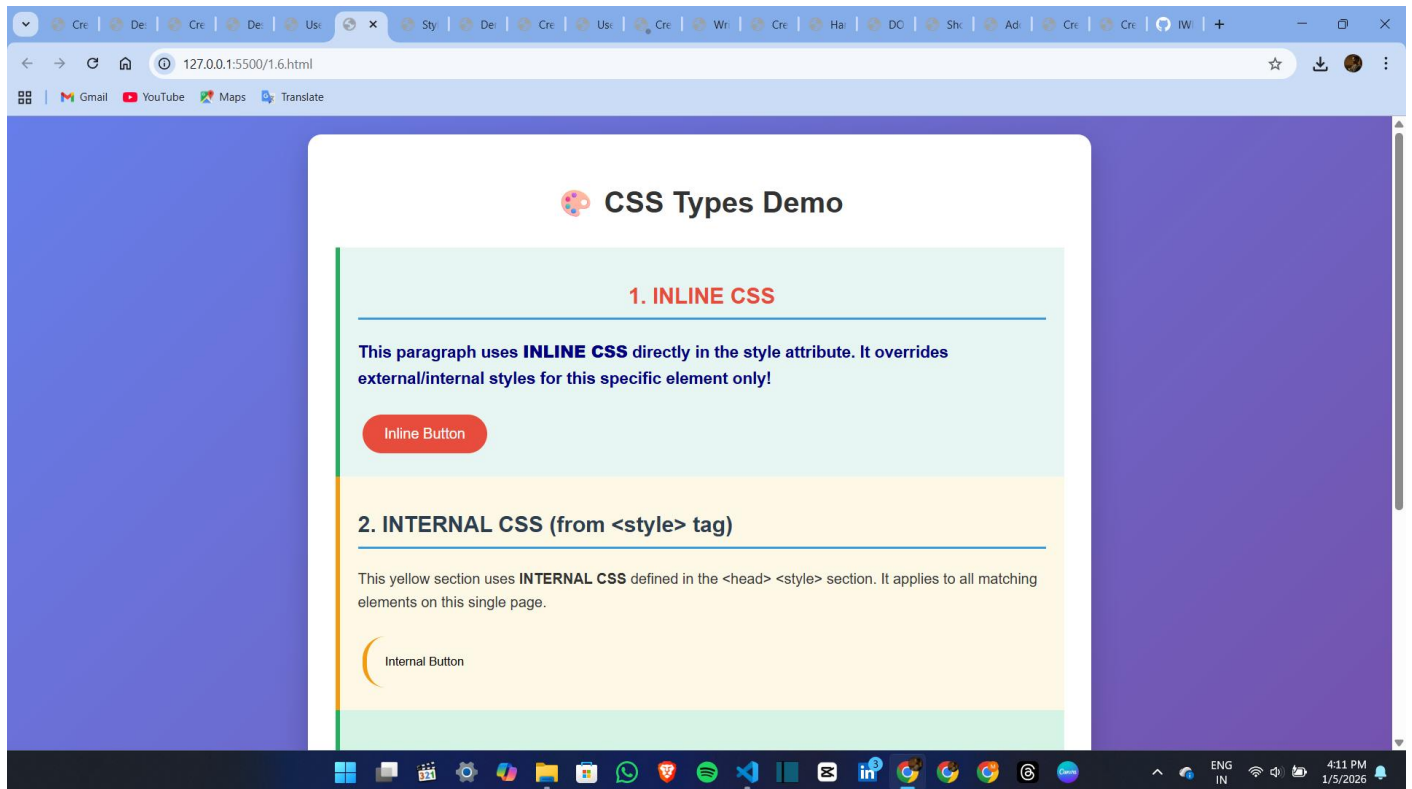
A screenshot of a web browser displaying a registration form. The browser's address bar shows the URL `127.0.0.1:5500/1.4.html`. The form includes a text input field labeled "Other" with a small circle icon above it. Below this is a section titled "Programming Languages (Select multiple):" with four checkboxes: "HTML/CSS", "JavaScript", "C Programming", and "Python". The "C Programming" checkbox is selected. Below the checkboxes is a "Select Course:" dropdown menu with the placeholder text "Choose your course...". Underneath the dropdown is a "Comments:" section with a text area containing the placeholder text "Tell us about your learning goals...". At the bottom of the form are two buttons: "Submit Registration" (green) and "Reset Form" (red). Below the buttons, a note reads: "Test it: Submit sends to httpbin.org (shows your data). Reset clears everything. Perfect for your HTML/CSS learning!". The browser's taskbar at the bottom shows various application icons and the system clock indicating 4:11 PM on 1/5/2026.

A screenshot of a web browser displaying a "Student Registration Form". The browser's address bar shows the URL `127.0.0.1:5500/1.4.html`. The form has a title "Student Registration Form". It includes a "Full Name:" label followed by a text input field with the placeholder text "Enter your full name". Below this is an "Email:" label followed by a text input field containing the email address "student@example.com". The "Gender" section contains three radio buttons: "Male" (which is selected), "Female", and "Other". Below the gender section is a section titled "Programming Languages (Select multiple):" with four checkboxes: "HTML/CSS", "JavaScript", "C Programming", and an unlabeled checkbox at the bottom. The "C Programming" checkbox is selected. The browser's taskbar at the bottom shows various application icons and the system clock indicating 4:11 PM on 1/5/2026.

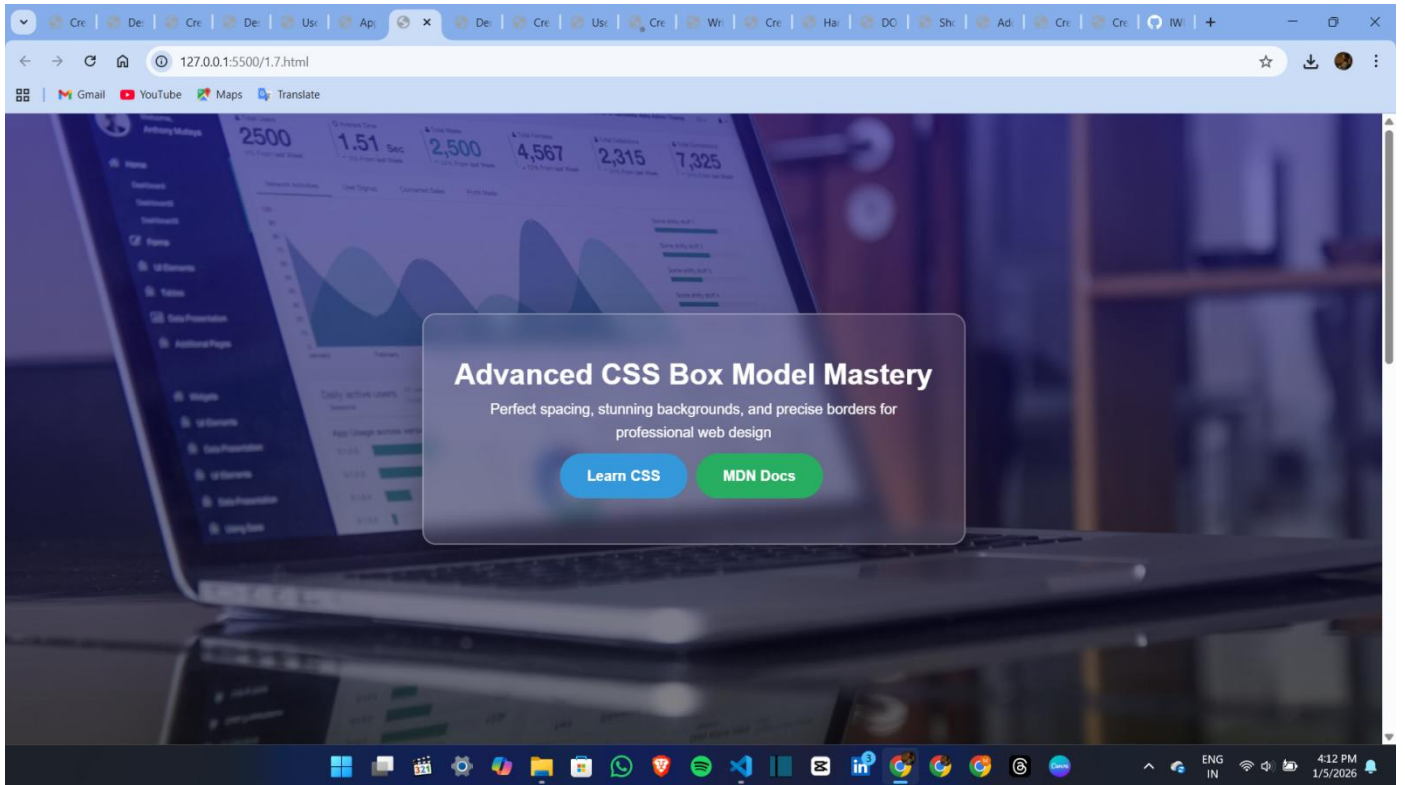
5. Use semantic elements (<article>, <section>, <nav>, <aside>, <footer>) to build a structured web page



6 Apply Inline, Internal, and External CSS styles to HTML elements



7. Style a web page using advanced CSS: Box Model, background images, borders, margins, and padding



8. Demonstrate the use of different CSS selectors (class, id, descendant, group, universal)

CSS Selectors - Complete Demo

1. Universal Selector (*) ★

`* { margin: 0; }` - Removes default margins/padding from ALL elements on page.
Applied to every HTML element automatically!

2. Class Selector (.demo-class) ●

All elements with `class="demo-class"` get green background + left border.

Multiple elements can share the same class!

3. ID Selector (#unique-id) ●

#unique-id targets ONLY this specific div. IDs must be unique!

Gradient background + dashed red border = ID selector power.

4. Group Selector (h3, p, ul) ●

Child Selector (section > div)

This yellow background is direct child of section.

This is also a direct child (yellow).

This paragraph is NOT a direct child (no yellow).

5. Descendant Selector (article p) ●

This paragraph inside article is italic + gray (descendant selector).

This nested paragraph is ALSO styled (all descendants).

6. Sibling Selectors

CreDeCreDeUsiApiSty

CreUsiCreWriCreHaiDCShiAdCreCreIW+

127.0.0.1:5500/1.8.html

GmailYouTubeMapsTranslate

This orange bold text follows h4 immediately (+ selector).

Adjacent spans are NOT affected.

General Sibling (~)

This ul and all following siblings get top border (~ selector).

- Second list item

Even paragraphs after h4 get affected by ~

7. Attribute Selector [type="text"]

Normal input:

Other input:

CSS Selector Priority

Order: Inline > ID > Class > Element > Universal

- ID (#unique-id)** = Highest priority
- Class (.demo-class)** = Medium priority
- Element (h2, p)** = Lower priority

ENG IN 4:12 PM 1/5/2026

9. Create a responsive web page using media queries and mobile-first design with flexible units

The screenshot displays a web browser window showing a responsive web page titled "Responsive Web Design". The page is designed to be mobile-first and responsive, using media queries and flexible units. The page is shown in a desktop view, but the content is designed to adapt to various screen sizes.

Responsive Web Design

Mobile-First + Media Queries + Flexible Units

Perfect on Every Device

Resize this page to see mobile → tablet → desktop layouts change live!

[MDN Responsive Guide](#)

Mobile First

Base styles target phones first (0px+).
Media queries add styles for larger screens using `min-width`.

- Flexible `%/rem/em` units
- Flexbox for layout
- Real Unsplash hero image

[freeCodeCamp Tutorial](#)

Breakpoints Used

Standard industry breakpoints:

- 768px: Tablets (Portrait)
- 1024px: Small Desktop
- 1440px: Large Desktop

[CSS-Tricks Guide](#)

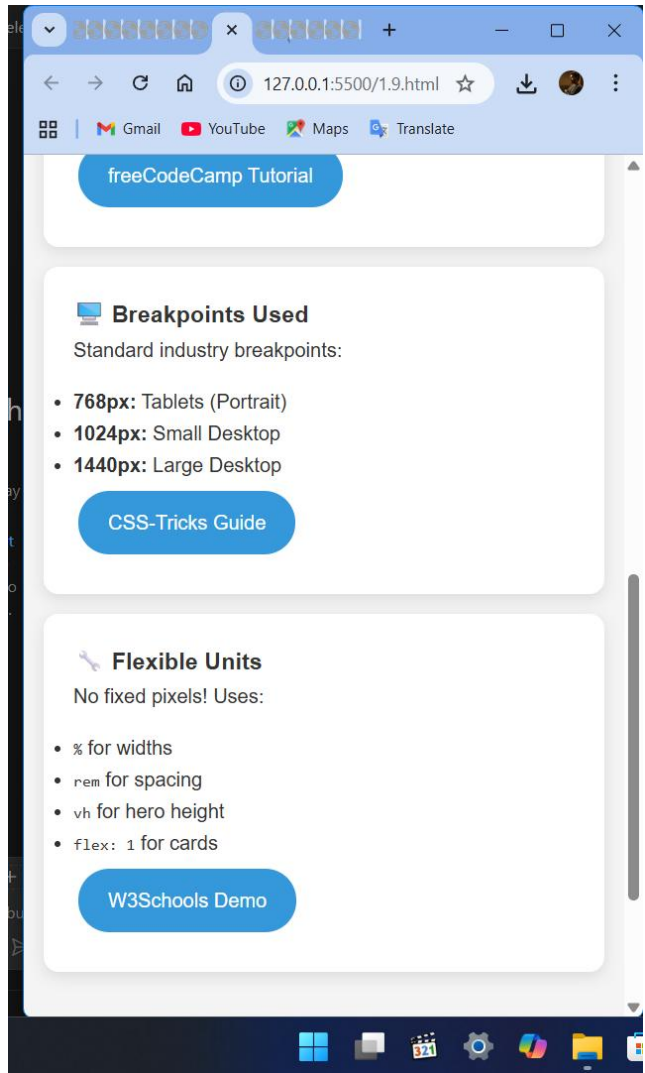
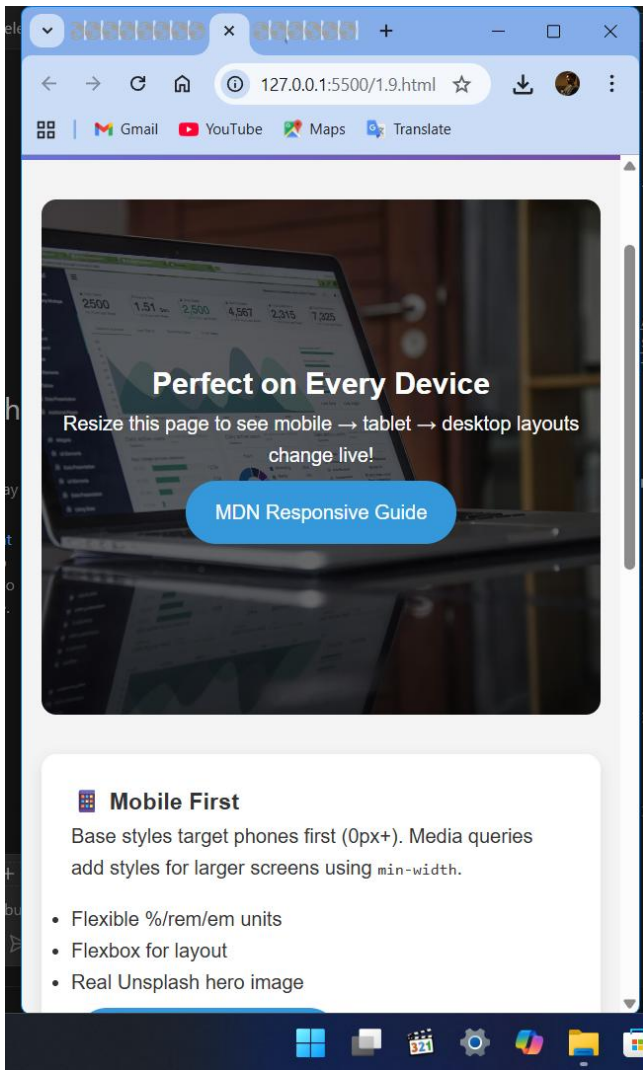
Flexible Units

No fixed pixels! Uses:

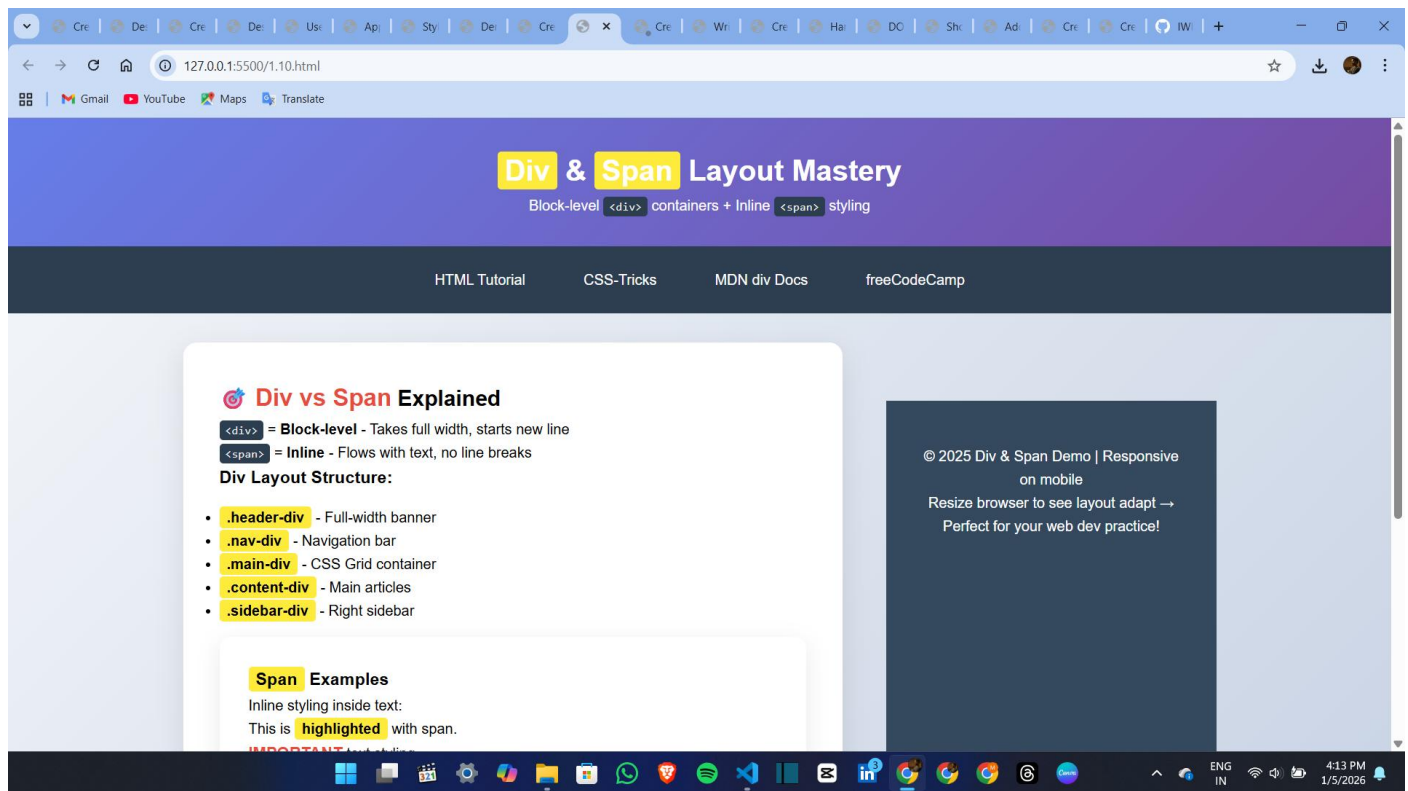
- `%` for widths
- `rem` for spacing
- `vh` for hero height
- `flex: 1` for cards

[W3Schools Demo](#)

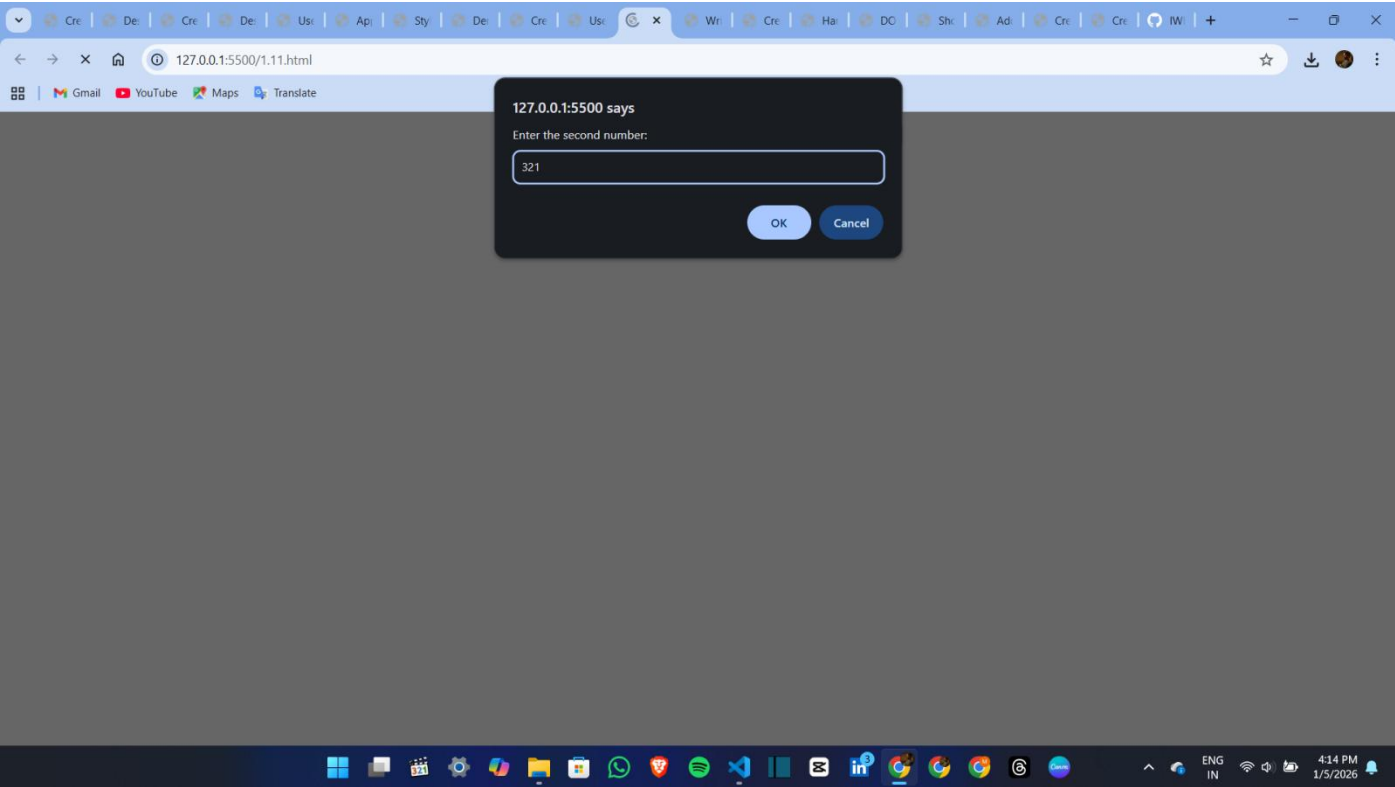
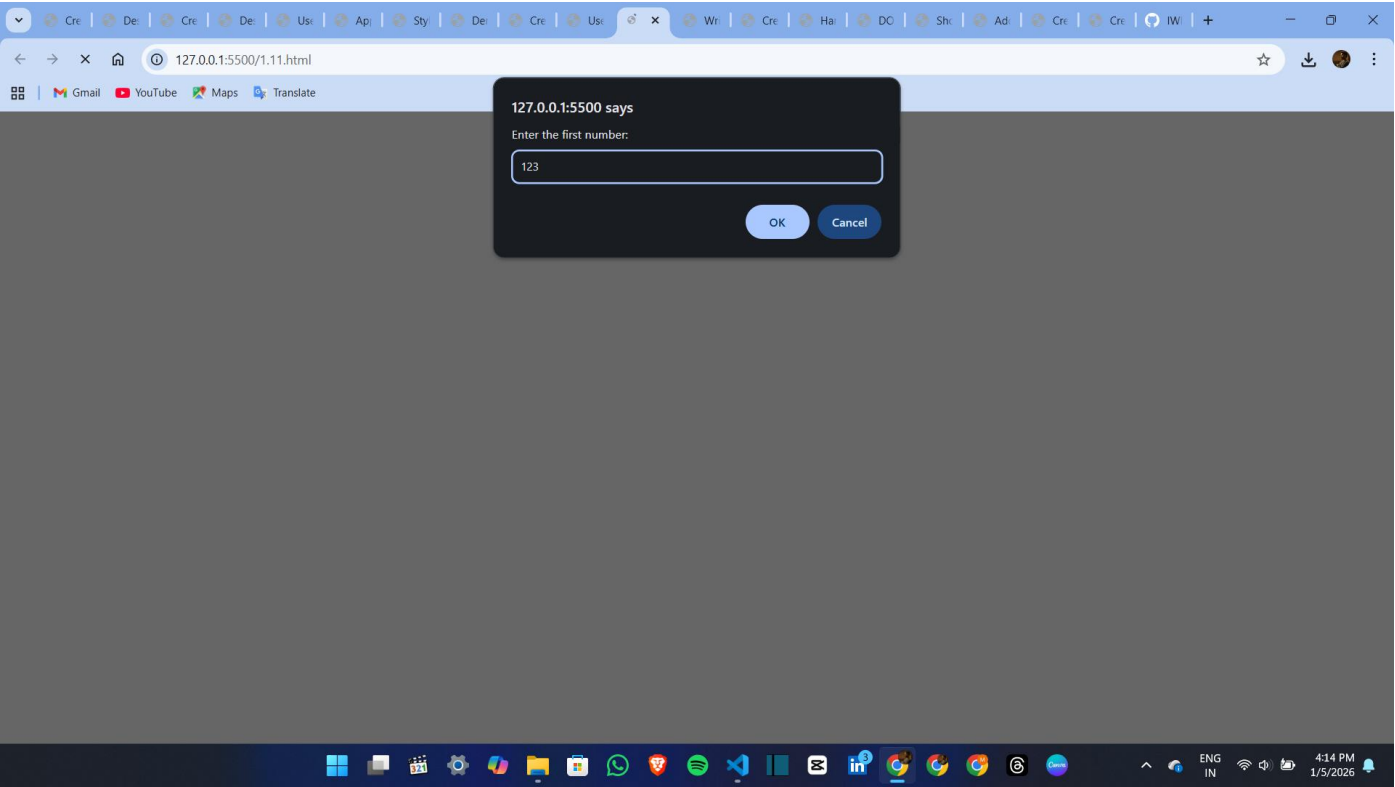
© 2025 Responsive Demo | All links work! | Built with Mobile-First CSS
Resize browser → Watch layout change live!

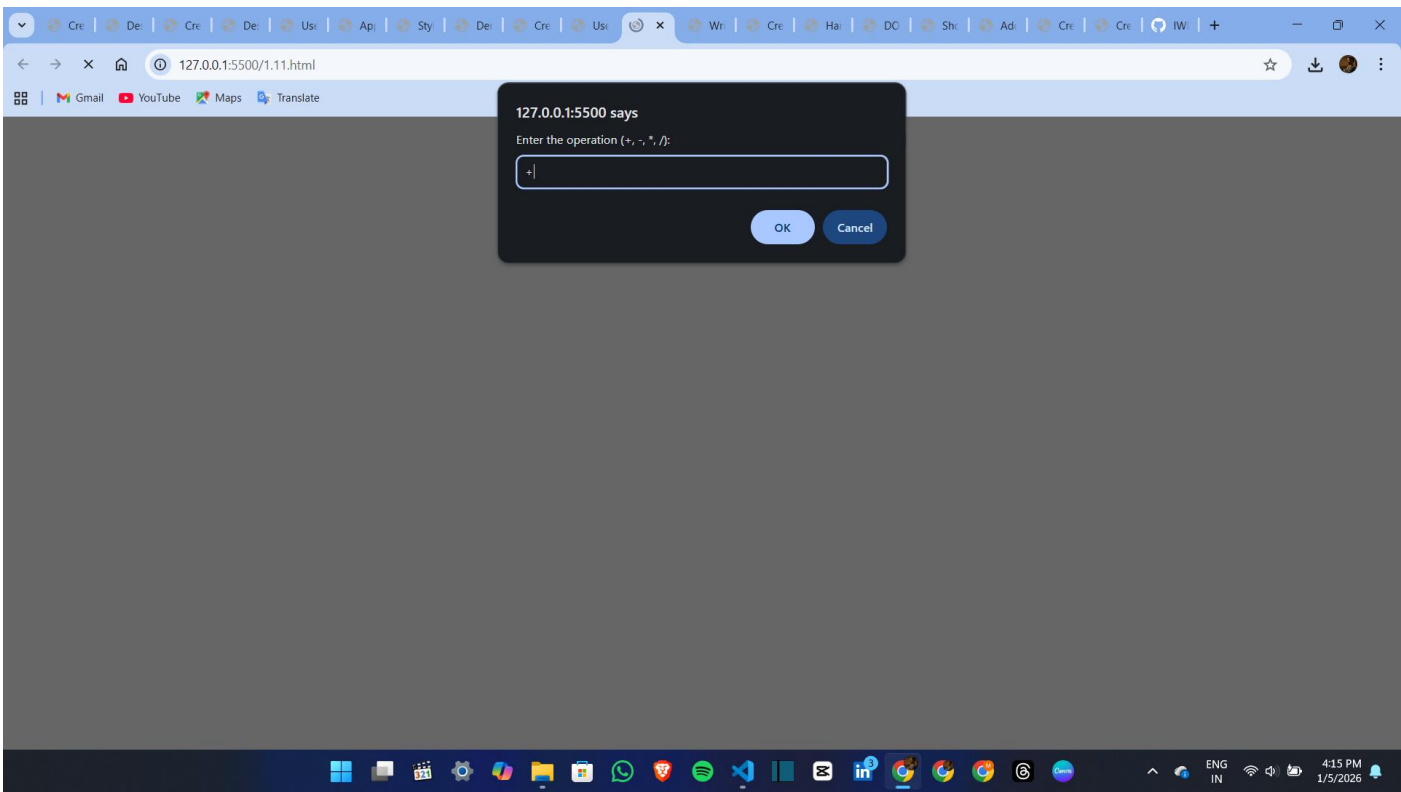
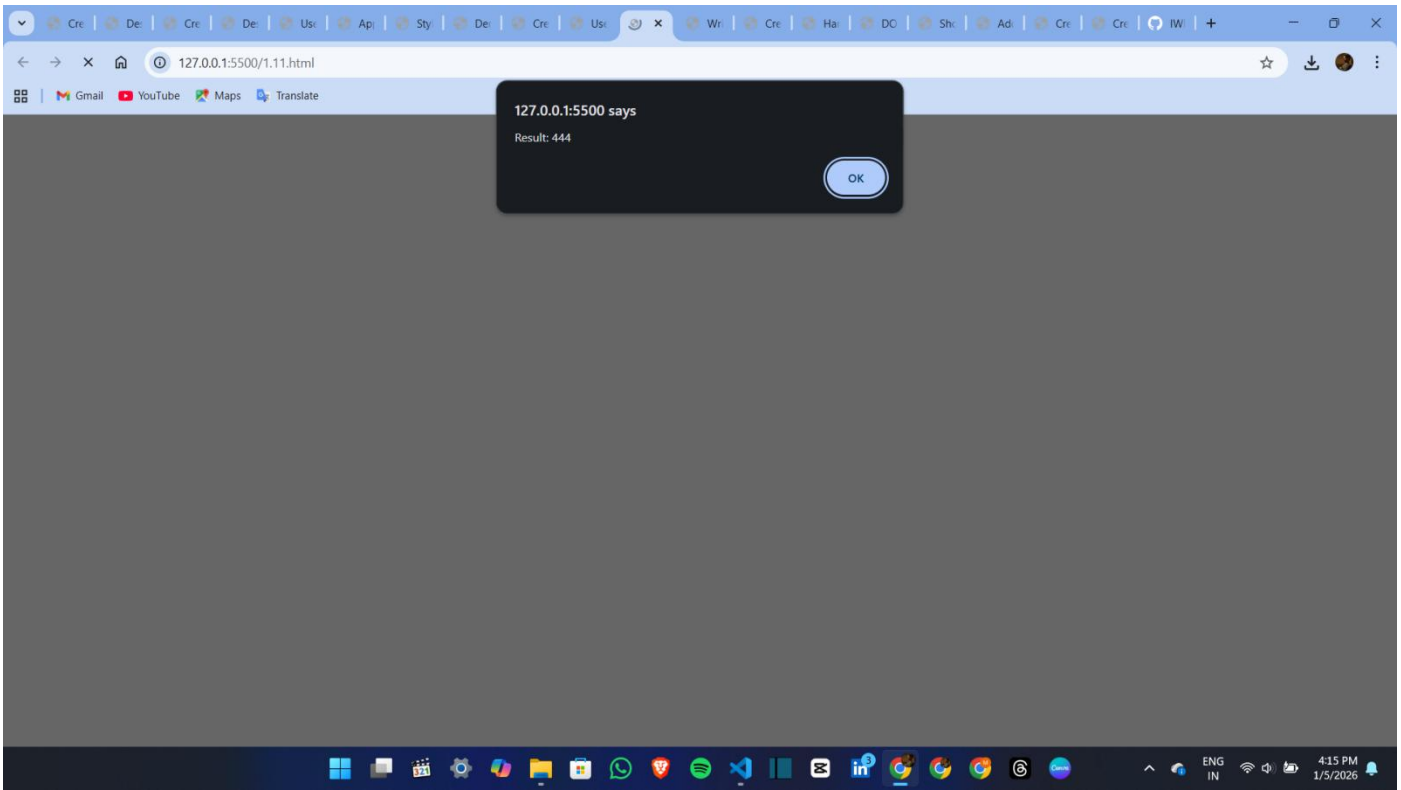


10. Use div and span elements for layout and apply styling with CSS

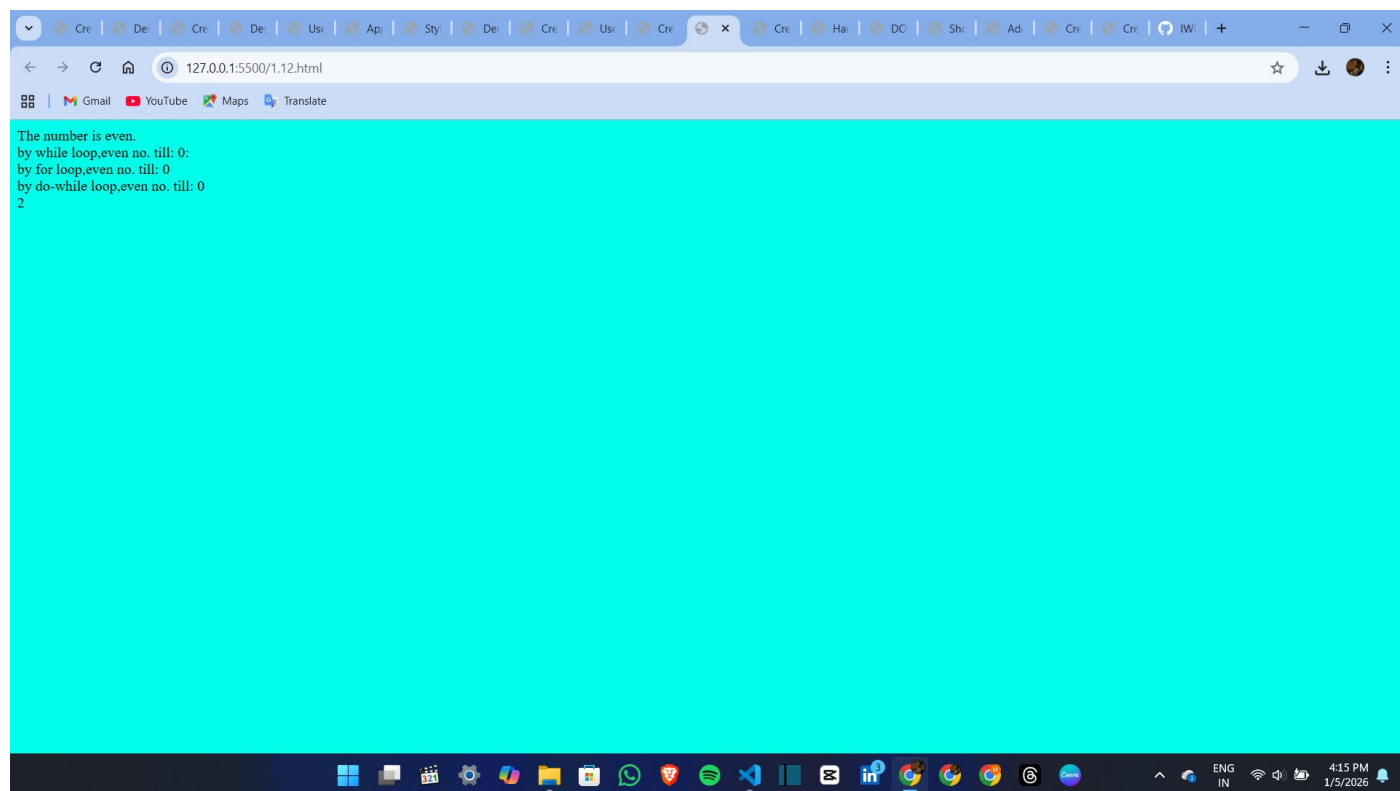


11. Create a JavaScript-enabled web page that performs basic arithmetic operations using input from users

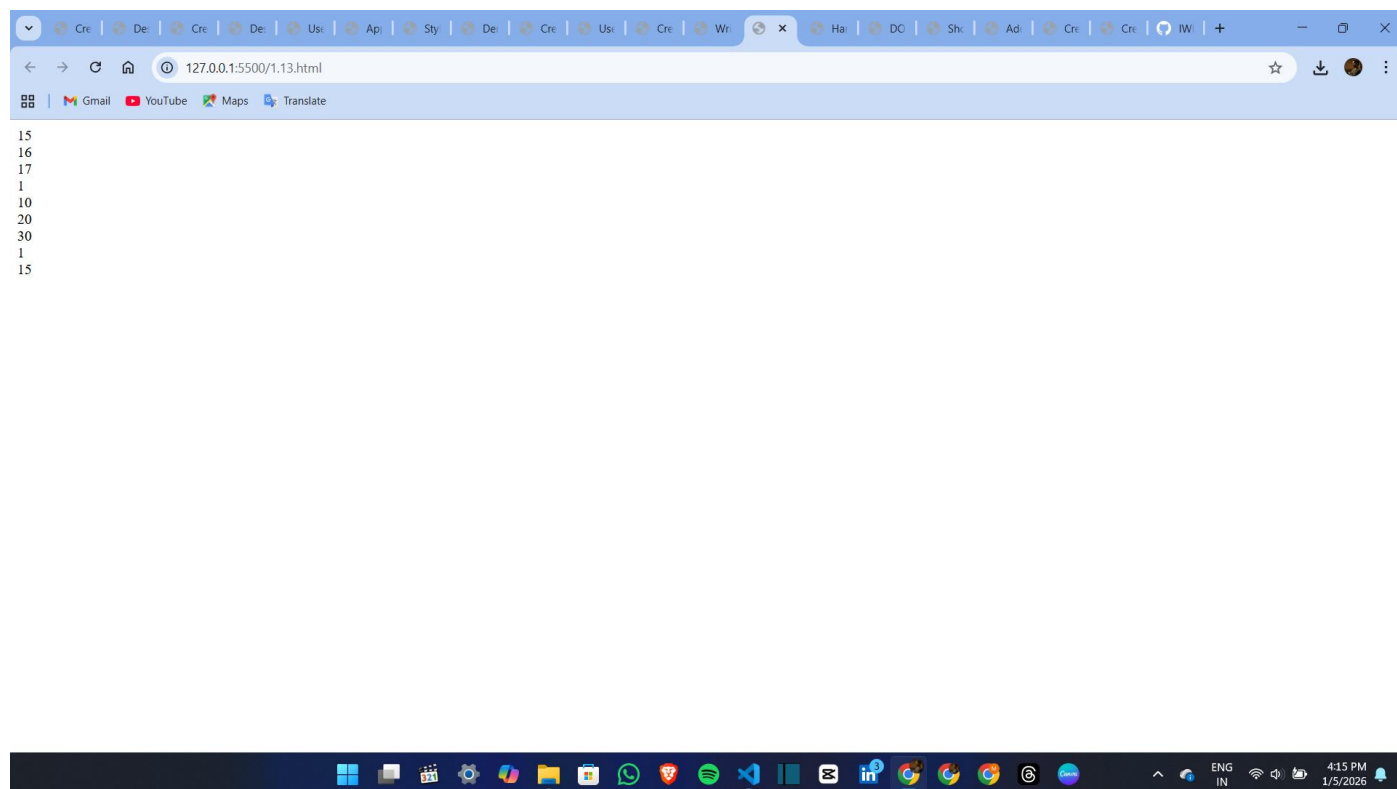




12. Write JavaScript to demonstrate conditional statements and looping (for, while, do...while)



13. Create and invoke user-defined functions in JavaScript; use var, let, and const for scope demonstration



14. Handle HTML form validation using JavaScript events like onsubmit, oninput, and use alert() for feedback

Cre | De: | Cre | De: | Use | Api | Sty | De: | Cre | Use | Cre | Wri | Cre

127.0.0.1:5500/1.14.html

Gmail | YouTube | Maps | Translate

Student Registration Form

Name:

Mayank Dewangan

Email:

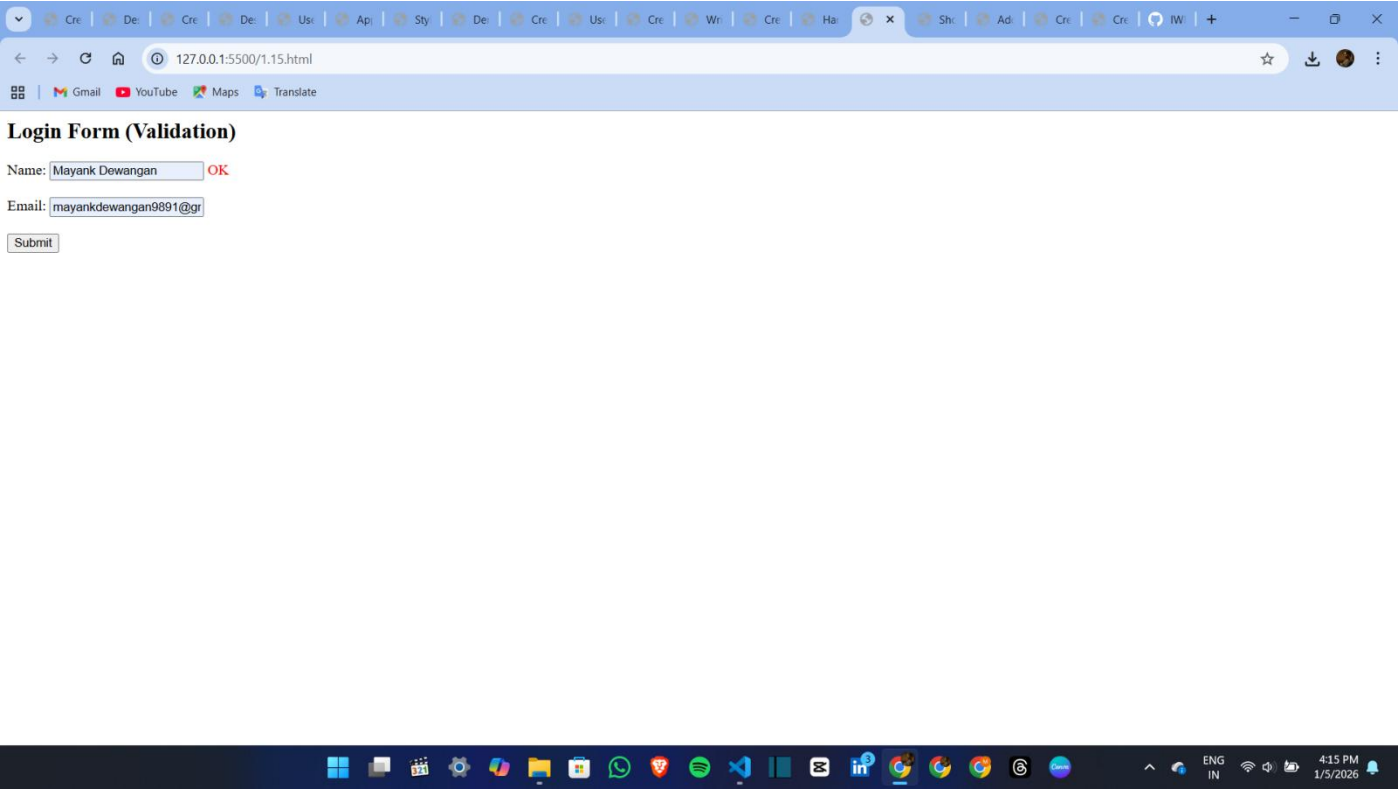
mayankdewangan9891@gr

Submit

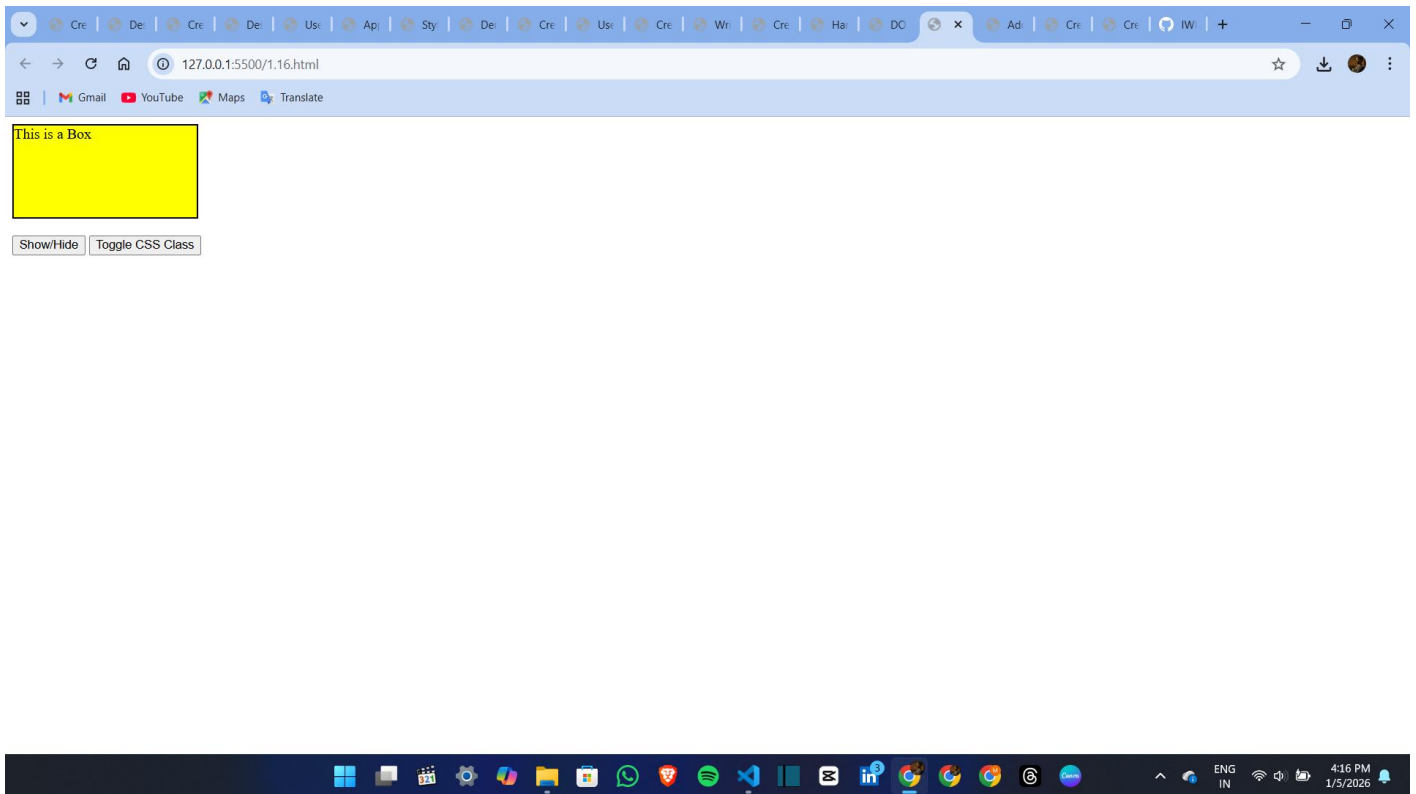
Windows | File Explorer | Settings | Edge | Chrome | WhatsApp | Spotify | VS Code | Telegram | Email | LinkedIn |

ENG IN | 4:15 PM 1/5/2026

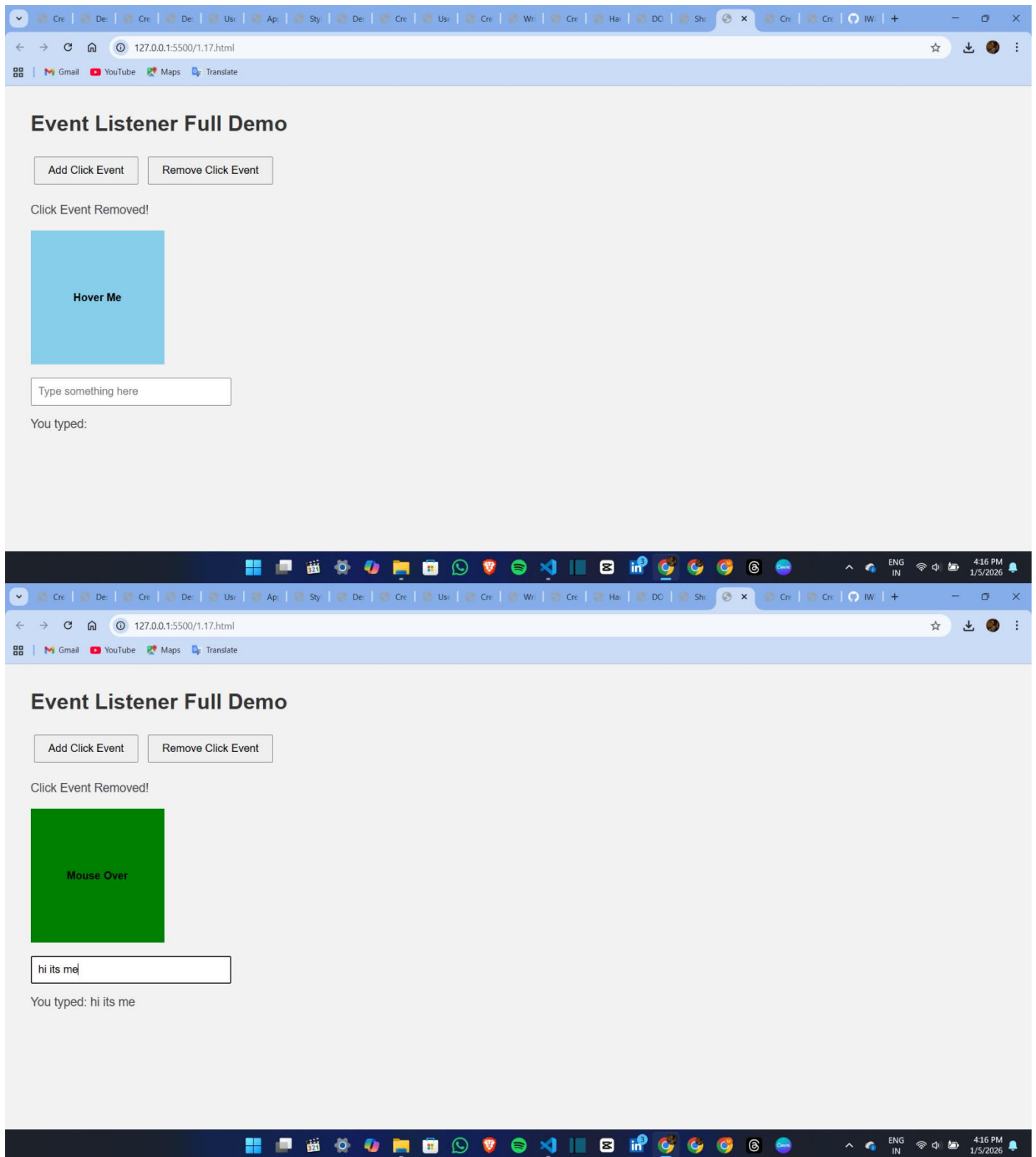
15. Demonstrate DOM manipulation using getElementById(), querySelector(), and innerHTML



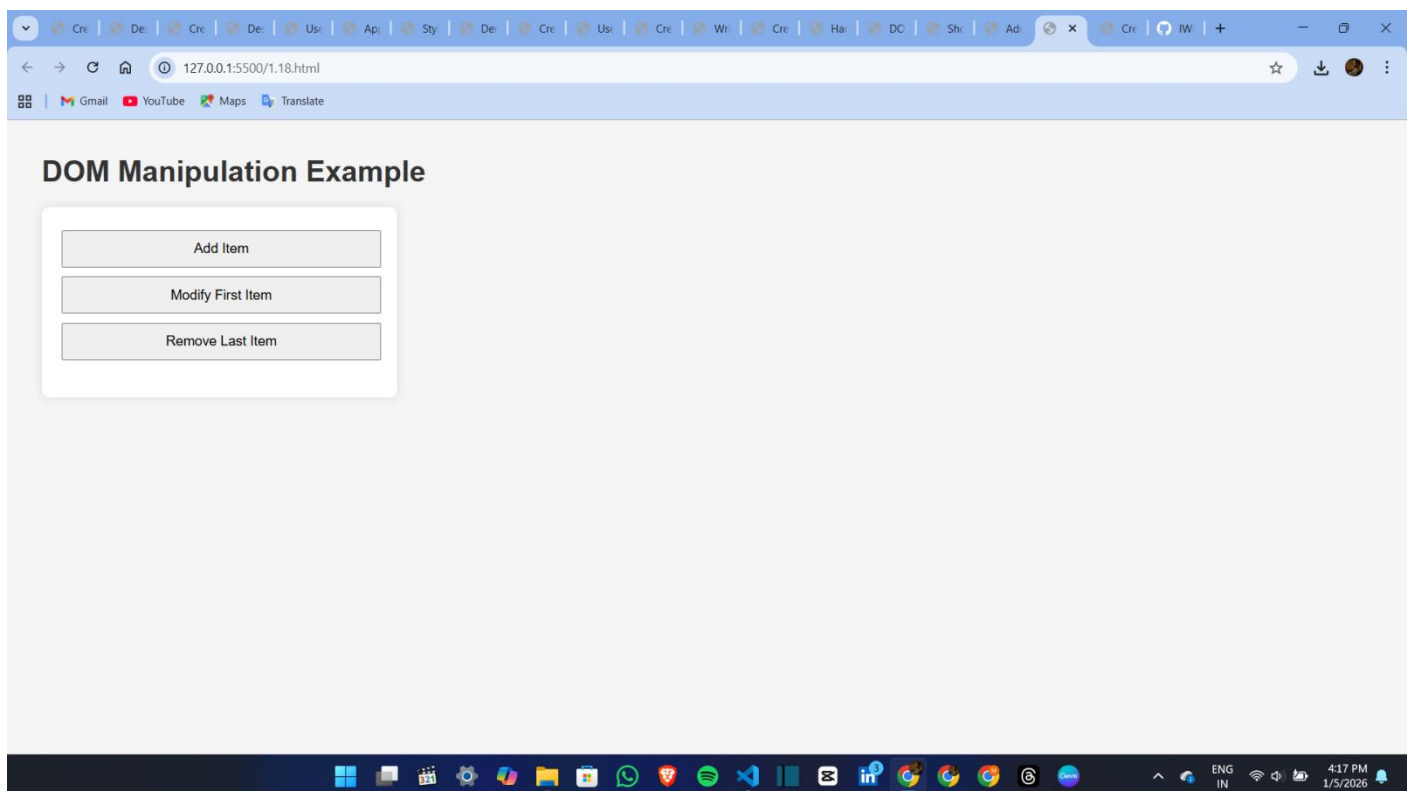
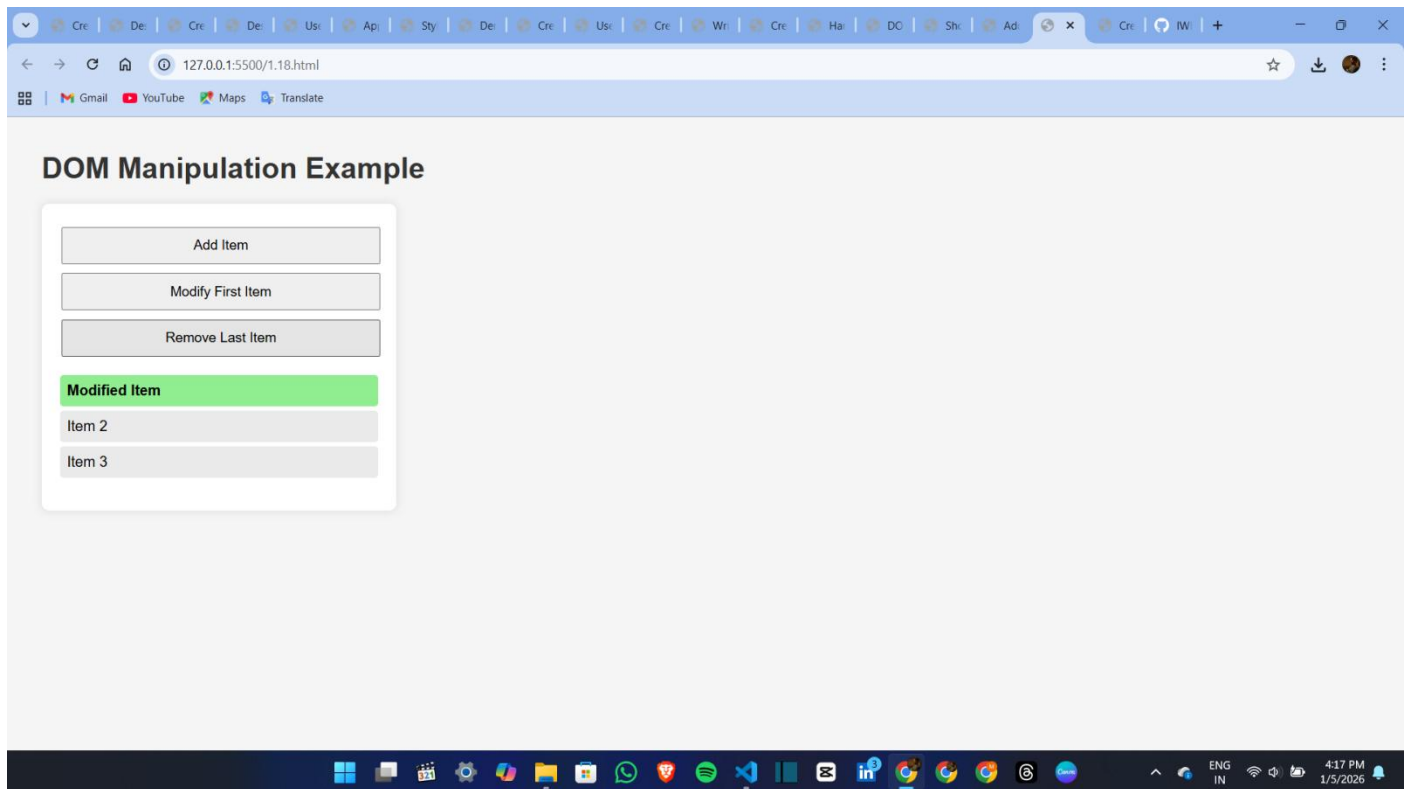
16. Show or hide HTML elements using JavaScript and toggle CSS classes dynamically



17. Add interactivity using event listeners (addEventListener, removeEventListener) for mouse/keyboard events



18. Create and manipulate elements using JavaScript: append, remove, or modify child nodes



19. Create a simple drawing using the <canvas> element: draw shapes and fill colors

